



CCS1303

Object-Oriented Programming

GROUP ASSIGNMENT

(SmartCare Hospital Management System)

- M.G. HIRUSHA PRAVEEN SULAKSHANA
CIT - 25 - 02 - 0334
- MALITHA HERATH
CIT - 25 - 02 - 0003
- MANULA GUNAWARDHANA
CIT - 25 - 02 - 0393
- THARUSHA NUWANGA
CIT - 25 - 02 - 0288

TABLE OF CONTENTS

1. INTRODUCTION.....	2
2.1 Operational & Business Requirements.....	3
2.2 System Assumptions.....	3
3.1 System Architecture.....	4
4.1 Encapsulation.....	6
4.3 Abstraction & Polymorphism.....	6
5. SPRING BOOT & DATABASE INTEGRATION.....	7
5.1 Database Connection Configuration.....	7
5.2 Persistence Layer Mapping.....	7
6.1 Spring REST Controller Structure.....	8
7. VALIDATION, EXCEPTION HANDLING & TESTING	9
7.1 Input Validation & Global Error Handling.....	9
8. CHALLENGES FACED & SOLUTIONS.....	10
9. CONCLUSION.....	11

1. INTRODUCTION

SmartCare Hospital is a growing healthcare institution in Sri Lanka providing services across departments like Cardiology, Pediatrics, Neurology, and Orthopedics. The primary objective of this project is to develop an enterprise-grade Object-Oriented software application using Java, Spring Boot, and MySQL.

Building upon the relational database created during the Database Systems module, this Spring Boot application automates clinical and administrative workflows. It handles patient registration, doctor scheduling, room allocations, medical treatments, lab test results, and billing calculations while ensuring data integrity and maintainability.

2. REQUIREMENT ANALYSIS & ASSUMPTIONS

2.1 Operational & Business Requirements

- **Patient Operations:** Complete registration, profile management, medical history tracking, and record deletion.
- **Doctor & Department Control:** Profile updates, consultation fee tracking, and department assignment.
- **Appointment Scheduling:** Prevent double-booking for the same doctor at identical dates and time slots.
- **Room & Admission Tracking:** Dynamic room allocation across General Ward, Private, and ICU categories.
- **Billing Calculations:** Automated aggregation of consultation, room occupancy, laboratory, and medicine fees.

2.2 System Assumptions

1. **Department Assignment:** A doctor belongs to one primary department, but a department employs multiple doctors.
2. **Schedule Integrity:** A doctor cannot have overlapping appointments at the same date and time slot.
3. **Room Availability:** Rooms can accommodate one patient per bed at a time and transition dynamically between 'Available' and 'Occupied' states.
4. **Prescriptions & Tests:** Treatments and lab requests must be issued by a registered doctor in the system.

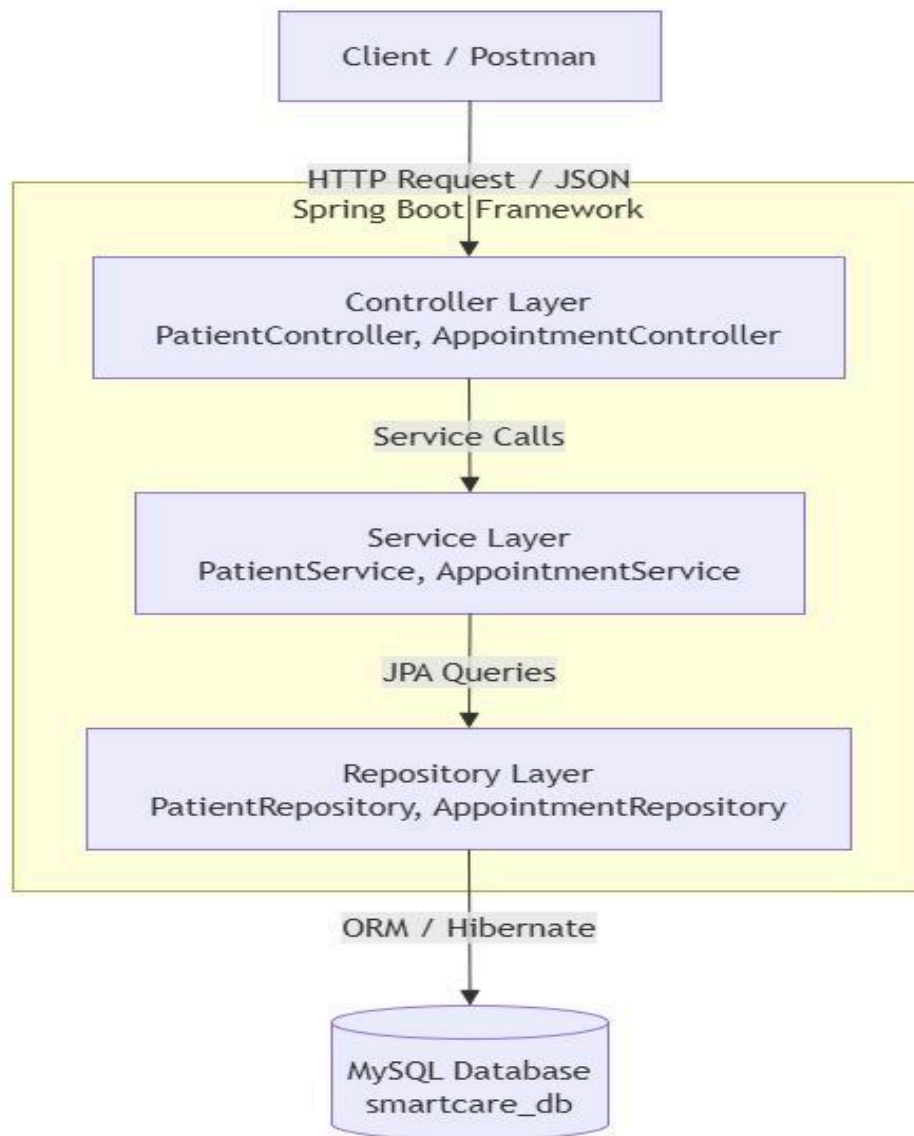
3. SYSTEM ARCHITECTURE & UML DESIGN DIAGRAM (ERD)

3.1 System Architecture

The application adheres to a standard 4-tier layered architecture:

- **Controller Layer:** Exposes RESTful endpoints, handles HTTP requests/responses, and parses input JSON payloads.
- **Service Layer:** Encapsulates core business logic, validation rules, and transaction control.
- **Repository Layer:** Extends Spring Data JPA repositories to handle database operations cleanly.
- **Entity Layer:** Maps Java domain objects directly to MySQL tables via Hibernate annotations.

Screenshot:



3.2 Domain Model Structure

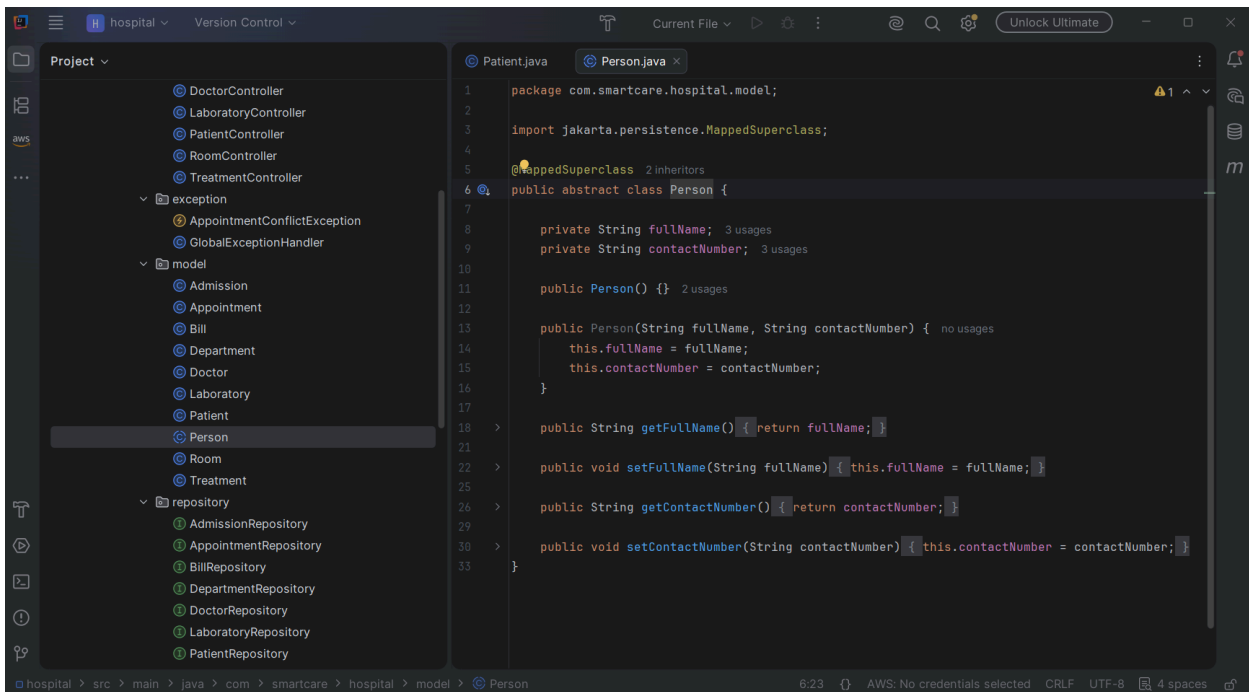
- **Abstract Class: Person** (contains id, fullName, contactNumber, address).
- **Subclasses: Patient and Doctor** extend Person.
- **Core Entities: Department, Appointment, Admission, Room, Treatment, LabTest, Bill.**
- **Interfaces: PaymentService** (Implemented by **CashPaymentService, CardPaymentService, OnlinePaymentService**).

4. CORE OOP CONCEPTS IMPLEMENTATION

4.1 Encapsulation

- All entity fields across the system are declared with **private** access modifiers to enforce strict data hiding. Direct access to field state from outside the class is prevented, and mutations or retrievals are controlled strictly through public getter and setter methods.

 **SCREENSHOT** : Code snippet from Person or Patient entity showing private fields alongside public getters and setters.

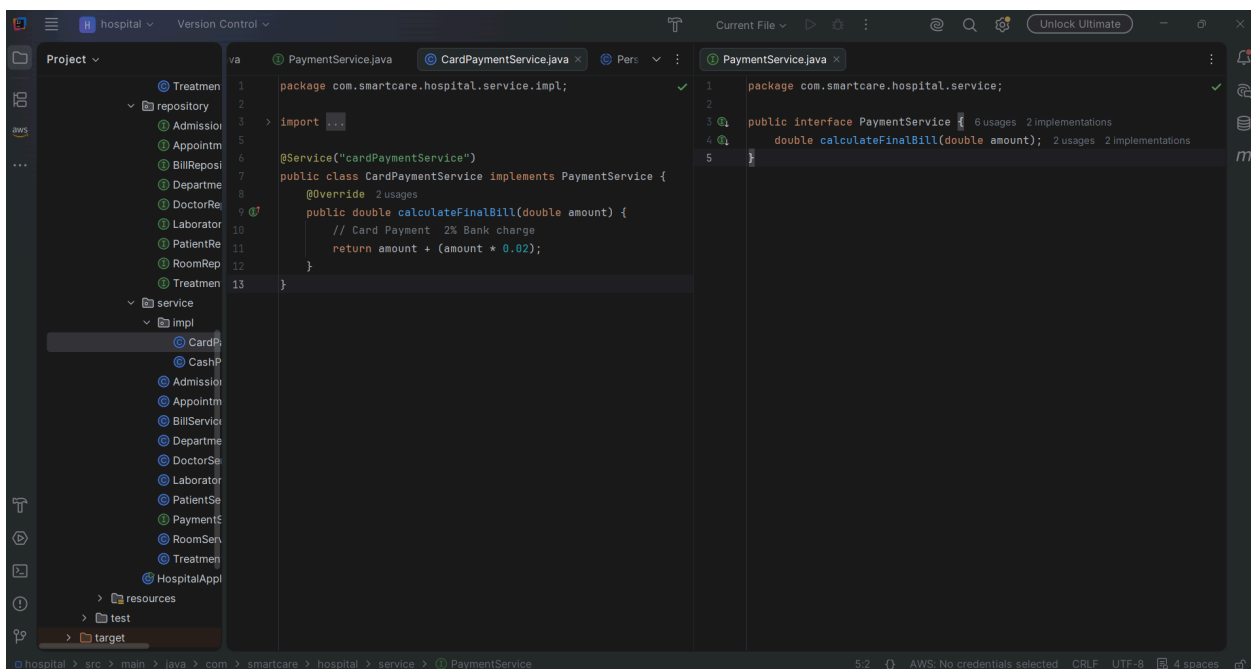


```
1 package com.smartcare.hospital.model;
2
3 import jakarta.persistence.MappedSuperclass;
4
5 @MappedSuperclass
6 public abstract class Person {
7
8     private String fullName;
9     private String contactNumber;
10
11     public Person() {}
12
13     public Person(String fullName, String contactNumber) {
14         this.fullName = fullName;
15         this.contactNumber = contactNumber;
16     }
17
18     public String getFullName() { return fullName; }
19
20     public void setFullName(String fullName) { this.fullName = fullName; }
21
22     public String getContactNumber() { return contactNumber; }
23
24     public void setContactNumber(String contactNumber) { this.contactNumber = contactNumber; }
25 }
```

4.3 Abstraction & Polymorphism

- The PaymentService interface abstracts payment handling logic, allowing multiple concrete implementations (CashPaymentServiceImpl, CardPaymentServiceImpl, OnlinePaymentServiceImpl) to process payments polymorphically based on the selected channel at runtime.

 SCREENSHOT : Code snippet showing PaymentService interface and one of its implementation classes (e.g., CardPaymentServiceImpl).



The screenshot displays an IDE with two code files open. The left pane shows the project structure with a 'service' package containing an 'impl' sub-package. The 'CardPaymentService.java' file is selected. The right pane shows the 'PaymentService.java' file. The 'PaymentService' interface is defined with a 'calculateFinalBill' method. The 'CardPaymentService' class implements this interface, adding a 2% bank charge to the final bill amount.

```
package com.smartcare.hospital.service.impl;

import ...

@Service("cardPaymentService")
public class CardPaymentService implements PaymentService {
    @Override
    public double calculateFinalBill(double amount) {
        // Card Payment 2% Bank charge
        return amount + (amount * 0.02);
    }
}
```

```
package com.smartcare.hospital.service;

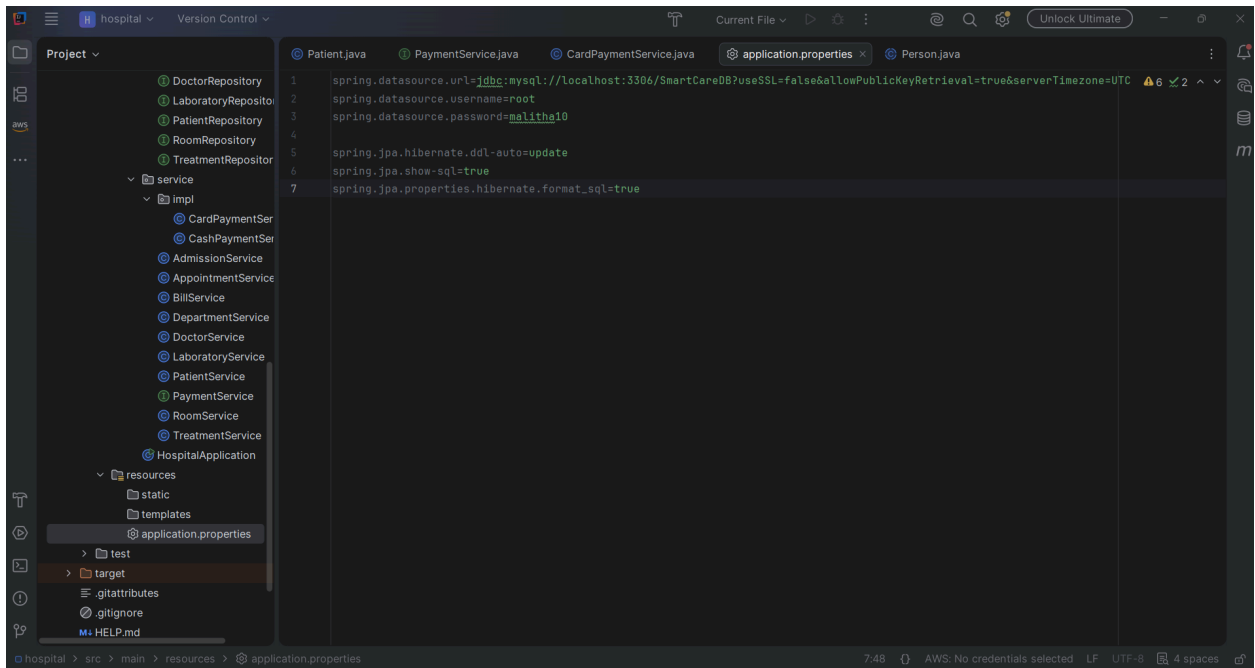
public interface PaymentService {
    double calculateFinalBill(double amount);
}
```


5. SPRING BOOT & DATABASE INTEGRATION

5.1 Database Connection Configuration

The application connects directly to the existing MySQL schema using Spring Data JPA. Properties define connection URLs, credentials, and Hibernate dialect settings.

 **SCREENSHOT** : Text editor image of the application.properties file showing MySQL connection settings.



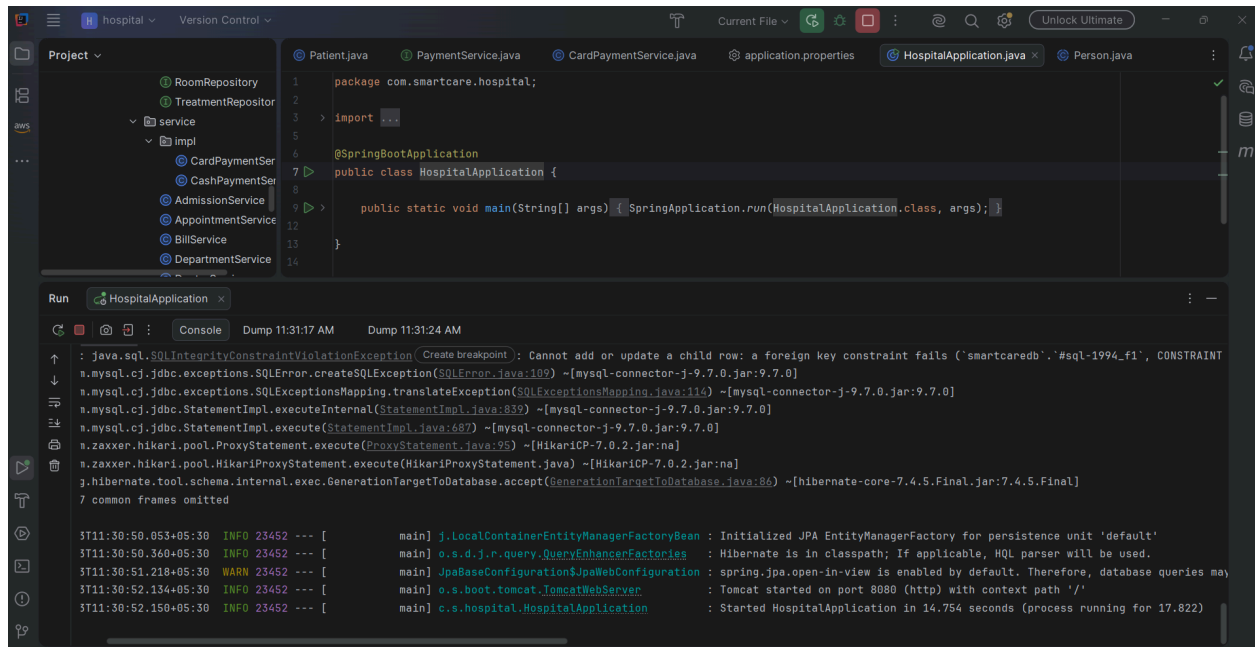
```
1 spring.datasource.url=jdbc:mysql://localhost:3306/SmartCare08?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
2 spring.datasource.username=root
3 spring.datasource.password=malitha10
4
5 spring.jpa.hibernate.ddl-auto=update
6 spring.jpa.show-sql=true
7 spring.jpa.properties.hibernate.format_sql=true
```

The screenshot shows an IDE window with the 'application.properties' file open. The file contains the following configuration for a MySQL database connection:

- Line 1: `spring.datasource.url=jdbc:mysql://localhost:3306/SmartCare08?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC`
- Line 2: `spring.datasource.username=root`
- Line 3: `spring.datasource.password=malitha10`
- Line 4: (empty line)
- Line 5: `spring.jpa.hibernate.ddl-auto=update`
- Line 6: `spring.jpa.show-sql=true`
- Line 7: `spring.jpa.properties.hibernate.format_sql=true`

The left sidebar shows the project structure with folders like 'resources', 'static', 'templates', 'test', and 'target'. The bottom status bar indicates the file is in the 'resources' directory.

SCREENSHOT : IDE Console terminal showing Spring Boot application startup logs and successful connection to MySQL database



```
package com.smartcare.hospital;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class HospitalApplication {

    public static void main(String[] args) { SpringApplication.run(HospitalApplication.class, args); }

}
```

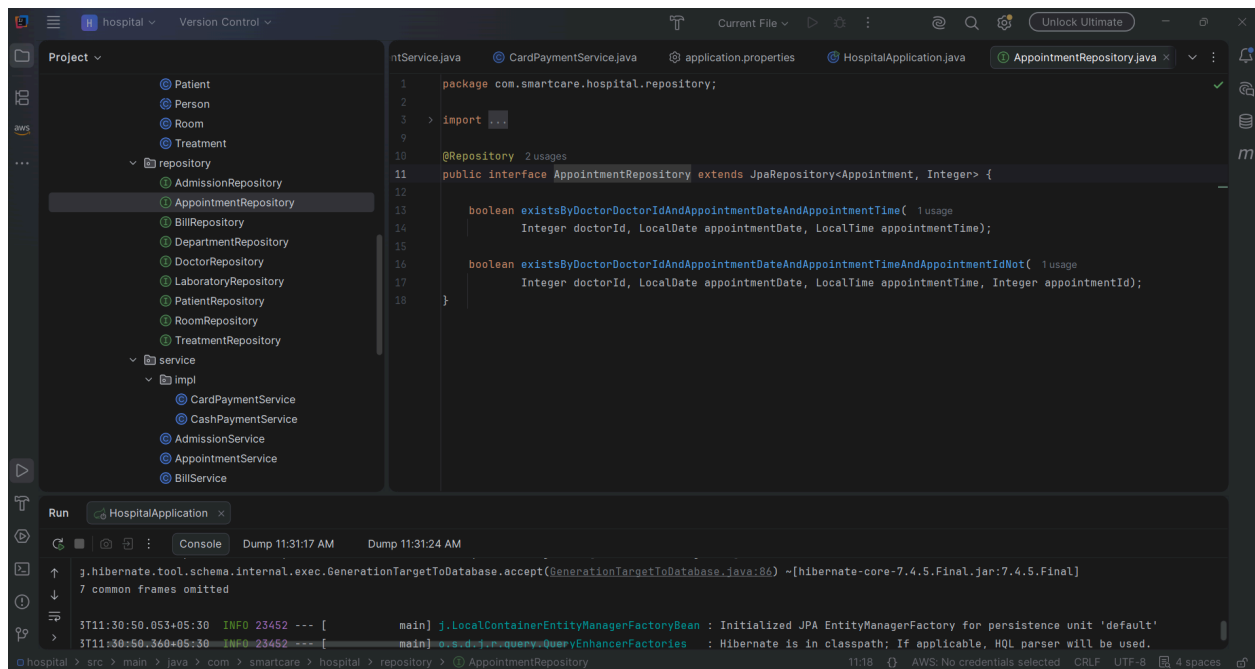
```
java.sql.SQLIntegrityConstraintViolationException: Cannot add or update a child row: a foreign key constraint fails ('smartcaredb'.'#sql-1994_f1', CONSTRAINT
n.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:109) ~[mysql-connector-j-9.7.0.jar:9.7.0]
n.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.java:114) ~[mysql-connector-j-9.7.0.jar:9.7.0]
n.mysql.cj.jdbc.StatementImpl.executeInternal(StatementImpl.java:839) ~[mysql-connector-j-9.7.0.jar:9.7.0]
n.mysql.cj.jdbc.StatementImpl.execute(StatementImpl.java:687) ~[mysql-connector-j-9.7.0.jar:9.7.0]
n.zaxxer.hikari.pool.ProxyStatement.execute(ProxyStatement.java:95) ~[HikariCP-7.0.2.jar:na]
n.zaxxer.hikari.pool.HikariProxyStatement.execute(HikariProxyStatement.java) ~[HikariCP-7.0.2.jar:na]
j.hibernate.tool.schema.internal.exec.GenerationTargetToDatabase.accept(GenerationTargetToDatabase.java:86) ~[hibernate-core-7.4.5.Final.jar:7.4.5.Final]
7 common frames omitted

3T11:30:50.053+05:30 INFO 23452 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
3T11:30:50.360+05:30 INFO 23452 --- [main] o.s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.
3T11:30:51.218+05:30 WARN 23452 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may
3T11:30:52.134+05:30 INFO 23452 --- [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
3T11:30:52.150+05:30 INFO 23452 --- [main] c.s.hospital.HospitalApplication : Started HospitalApplication in 14.754 seconds (process running for 17.822)
```

5.2 Persistence Layer Mapping

Spring Data JPA interface repositories abstract complex SQL database interactions, enabling standard CRUD operations and custom query execution without boiler-plate code.

SCREENSHOT : Code snippet of AppointmentRepository.



The screenshot displays an IDE interface with the following components:

- Project Explorer:** Shows a project structure with packages like `Person`, `Room`, `Treatment`, `repository` (containing `AdmissionRepository`, `AppointmentRepository`, `BillRepository`, `DepartmentRepository`, `DoctorRepository`, `LaboratoryRepository`, `PatientRepository`, `RoomRepository`, and `TreatmentRepository`), and `service` (containing `impl` with `CardPaymentService`, `CashPaymentService`, `AdmissionService`, `AppointmentService`, and `BillService`).
- Code Editor:** Shows the `AppointmentRepository.java` file with the following code:

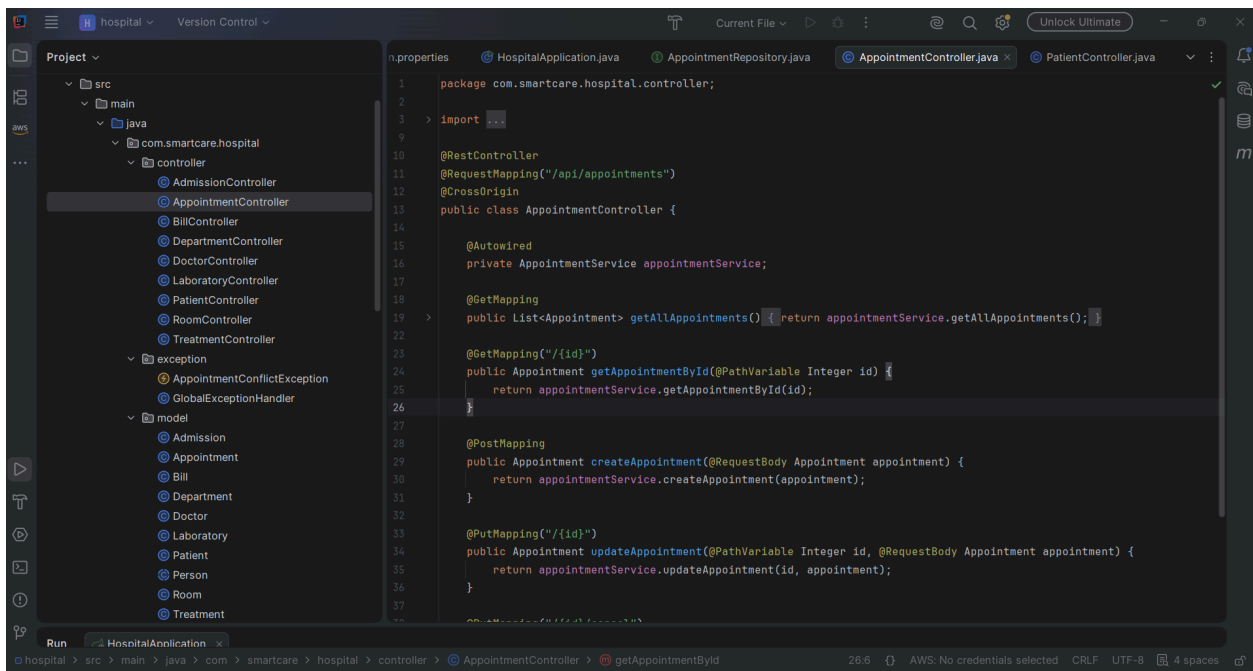
```
1 package com.smartcare.hospital.repository;
2
3 > import ...
4
5
6
7
8
9
10 @Repository 2 usages
11 public interface AppointmentRepository extends JpaRepository<Appointment, Integer> {
12
13
14     boolean existsByDoctorDoctorIdAndAppointmentDateAndAppointmentTime( 1 usage
15         Integer doctorId, LocalDate appointmentDate, LocalTime appointmentTime);
16
17     boolean existsByDoctorDoctorIdAndAppointmentDateAndAppointmentTimeAndAppointmentIdNot( 1 usage
18         Integer doctorId, LocalDate appointmentDate, LocalTime appointmentTime, Integer appointmentId);
19 }
```
- Run Console:** Shows the output of the application run, including logs from Hibernate and JPA. The logs indicate that the JPA EntityManagerFactory was initialized and that Hibernate is in the classpath.

6. CONTROLLER LAYER & REST API DOCUMENTATION

6.1 Spring REST Controller Structure

Controllers handle HTTP requests, invoke corresponding service layer business functions, and return appropriate HTTP response status codes and JSON payloads.

 SCREENSHOT : Code snippet of AppointmentController or PatientController showing REST mapping annotations.



```
1 package com.smartcare.hospital.controller;
2
3 import ...
4
5 @RestController
6 @RequestMapping("/api/appointments")
7 @CrossOrigin
8 public class AppointmentController {
9
10     @Autowired
11     private AppointmentService appointmentService;
12
13     @GetMapping
14     public List<Appointment> getAllAppointments() { return appointmentService.getAllAppointments(); }
15
16     @GetMapping("/{id}")
17     public Appointment getAppointmentById(@PathVariable Integer id) {
18         return appointmentService.getAppointmentById(id);
19     }
20
21     @PostMapping
22     public Appointment createAppointment(@RequestBody Appointment appointment) {
23         return appointmentService.createAppointment(appointment);
24     }
25
26     @PutMapping("/{id}")
27     public Appointment updateAppointment(@PathVariable Integer id, @RequestBody Appointment appointment) {
28         return appointmentService.updateAppointment(id, appointment);
29     }
30 }
```

6.2 REST Endpoints Overview

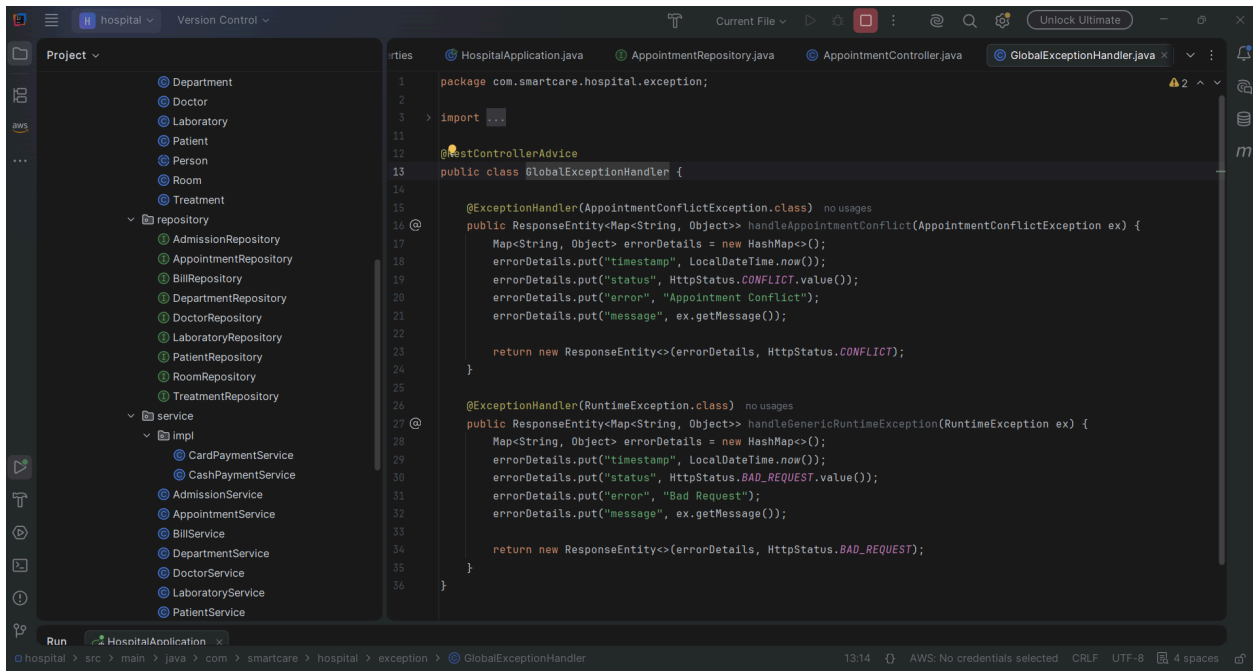
Module	Method	Endpoint	Description
Patient	POST	/api/patients	Registers a new patient record
Patient	GET	/api/patients/{id}	Retrieves detailed patient information
Doctor	GET	/api/doctors/search	Searches doctors by specialization or department
Appointment	POST	/api/appointments	Schedules appointment after checking conflicts
Admission	POST	/api/admissions	Admits patient and marks room as occupied
Billing	POST	/api/billing/generate/{patientId}	Calculates charges and generate bill

7. VALIDATION, EXCEPTION HANDLING & TESTING

7.1 Input Validation & Global Error Handling

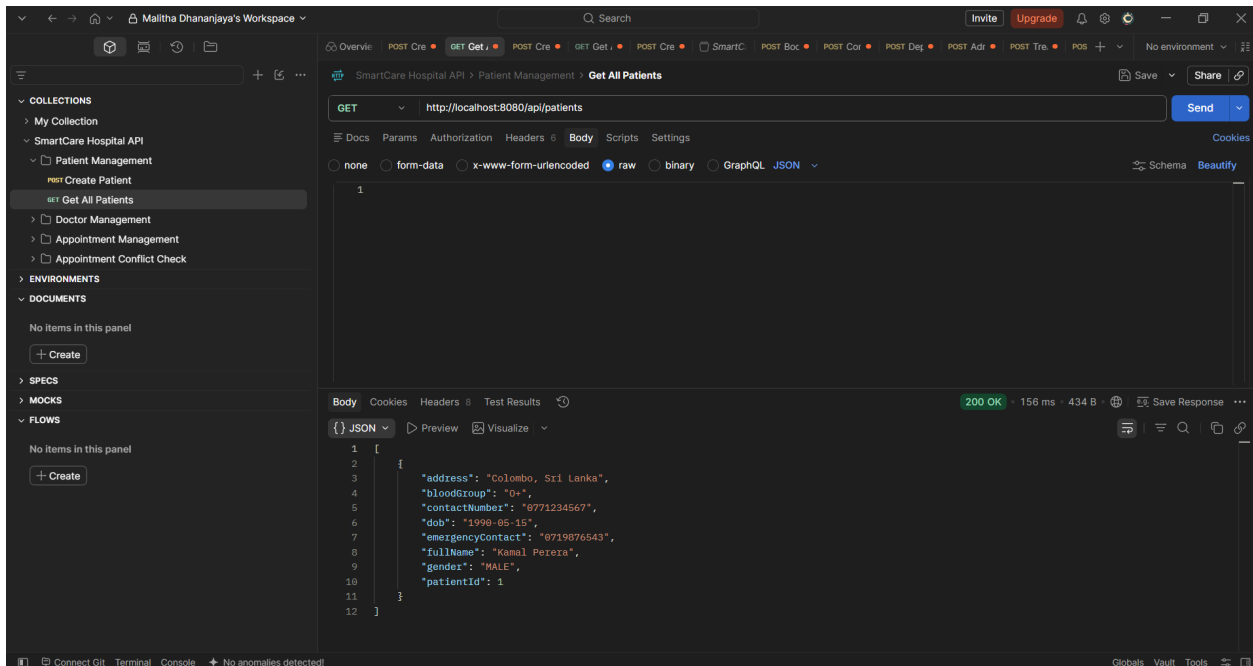
- The application uses global exception handling (@ControllerAdvice) to capture validation errors and custom exceptions, transforming them into clean, structured JSON error responses.

 SCREENSHOT : Code snippet of GlobalExceptionHandler class showing @ExceptionHandler methods.

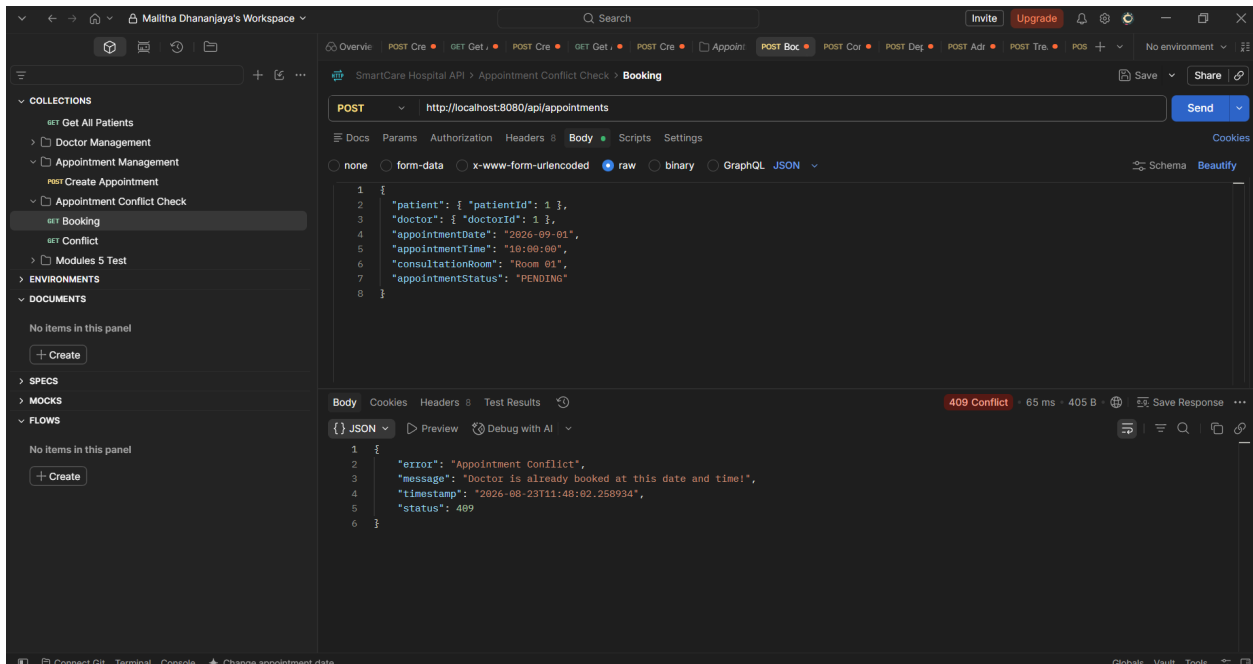


```
1 package com.smartcare.hospital.exception;
2
3 import java.util.HashMap;
4 import java.util.Map;
5
6 @ControllerAdvice
7 public class GlobalExceptionHandler {
8
9     @ExceptionHandler({AppointmentConflictException.class})
10     public ResponseEntity<Map<String, Object>> handleAppointmentConflict(AppointmentConflictException ex) {
11         Map<String, Object> errorDetails = new HashMap<>();
12         errorDetails.put("timestamp", LocalDateTime.now());
13         errorDetails.put("status", HttpStatus.CONFLICT.value());
14         errorDetails.put("error", "Appointment Conflict");
15         errorDetails.put("message", ex.getMessage());
16
17         return new ResponseEntity<>(errorDetails, HttpStatus.CONFLICT);
18     }
19
20     @ExceptionHandler({RuntimeException.class})
21     public ResponseEntity<Map<String, Object>> handleGenericRuntimeException(RuntimeException ex) {
22         Map<String, Object> errorDetails = new HashMap<>();
23         errorDetails.put("timestamp", LocalDateTime.now());
24         errorDetails.put("status", HttpStatus.BAD_REQUEST.value());
25         errorDetails.put("error", "Bad Request");
26         errorDetails.put("message", ex.getMessage());
27
28         return new ResponseEntity<>(errorDetails, HttpStatus.BAD_REQUEST);
29     }
30 }
```

 SCREENSHOT : Postman execution showing a successful GET or POST response with HTTP 200 OK or 201 Created



 SCREENSHOT : Postman execution showing an intentional error trigger (e.g., duplicate appointment conflict) returning HTTP 400 Bad Request.



8. CHALLENGES FACED & SOLUTIONS

- **Appointment Clashes**

Handled double-booking issues by adding repository validation queries (existsByDoctorIdAndAppointmentDateAndAppointmentTime) inside transactional service methods

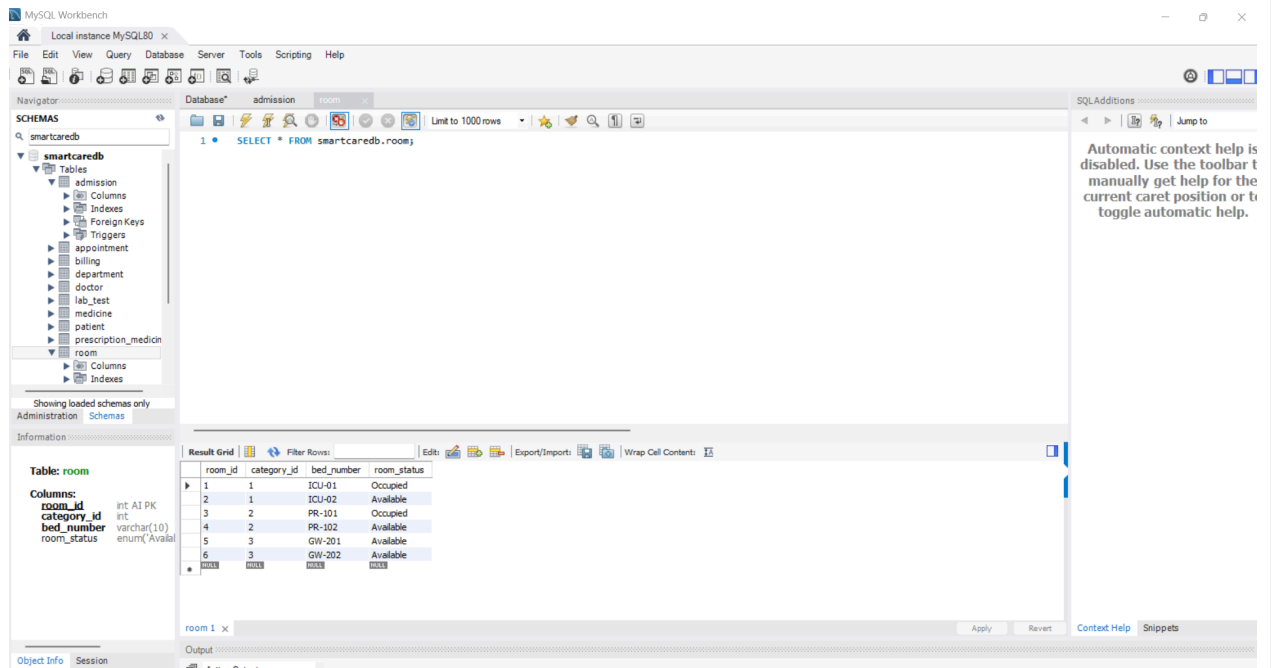
- **Circular JSON References**

Jackson serialization infinite loops occurred due to bidirectional relationships between entities. Resolved by introducing DTOs and applying @JsonIgnore annotations on inverse navigation fields.

- **Room State Synchronization**

Ensured room availability dynamically updates upon patient admission and discharge by chaining updates in service methods.

 SCREENSHOT : MySQL Workbench view showing updated Room states transitioning between 'Available' and 'Occupied'.



The screenshot displays the MySQL Workbench interface for a local instance of MySQL 8.0. The 'Navigator' pane on the left shows the 'smartcaredb' database with various tables and schemas. The 'Database' pane in the center shows the 'room' table selected, with a query 'SELECT * FROM smartcaredb.room;' entered. The 'Result Grid' pane on the right displays the data for the 'room' table, showing columns 'room_id', 'category_id', 'bed_number', and 'room_status'. The data is as follows:

room_id	category_id	bed_number	room_status
1	1	ICU-01	Occupied
2	1	ICU-02	Available
3	2	PR-101	Occupied
4	2	PR-102	Available
5	3	GW-201	Available
6	3	GW-202	Available

The 'Information' pane at the bottom left shows the table structure for 'room':

Table: room

Columns:

- room_id: int AI PK
- category_id: int
- bed_number: varchar(10)
- room_status: enum('Available', 'Occupied')

9. CONCLUSION

The SmartCare Hospital Management System backend effectively translates core Object-Oriented Programming concepts into an enterprise Spring Boot application. By integrating with MySQL via Spring Data JPA, the system delivers structured API services, enforces business validation rules, and maintains strong software design standards suitable for production extension.

10. REFERENCES

- <https://www.w3schools.com/MYSQL/default.asp>
- Spring Framework Documentation: <https://spring.io/projects/spring-boot>
- Oracle Java SE Documentation: <https://docs.oracle.com/en/java/>
- GeeksforGeeks DBMS Concepts: <https://www.geeksforgeeks.org/dbms/>
- MySQL 8.0 Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
- Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems*. Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*. McGraw-Hill.
- https://drive.google.com/drive/folders/18rSzEwyffbR3gLy3QkdZwfriA5N6bN_V?usp=sharing
- https://github.com/MalithaDhananjaya/CCS1303-Object_Oriented_Programming-Group_project

END OF REPORT

